

RISC VS CISC SIMULATION TOOL

Project Documentation

1. Introduction

Computer Organization and Architecture (COA) deals with the internal working of a computer system and how instructions are processed by the CPU. Two important instruction set architecture approaches are RISC (Reduced Instruction Set Computer) and CISC (Complex Instruction Set Computer).

This project is a simple RISC vs CISC Simulation Tool developed using Python and Tkinter. It visually compares how the same assembly instructions can be represented differently in RISC and CISC architectures. The simulator supports ADD, SUB, MUL and MOV instructions and displays instruction count, estimated clock cycles, execution sequences and a simple comparison.

2. Objective

- To understand the basic concept of RISC and CISC architectures.
- To simulate the execution of simple assembly instructions.
- To compare the number of instructions required by RISC and CISC.
- To compare their estimated clock cycles.
- To provide a simple graphical interface for learning the concept.

The project is an educational simulation and not a real processor emulator.

3. RISC and CISC

RISC

RISC stands for Reduced Instruction Set Computer. RISC uses a smaller set of simple instructions. In our simplified simulation, ADD is represented as LOAD → ADD → STORE, so it uses three instructions.

CISC

CISC stands for Complex Instruction Set Computer. CISC uses more complex instructions. In our simulation, ADD is represented simply as ADD.

4. Working of the Simulator

The simulator works in the following steps:

1. User enters assembly instructions.
2. The program reads each instruction.
3. The instruction type is identified.
4. A simplified RISC sequence is generated.
5. A simplified CISC sequence is generated.
6. Instruction count and estimated cycles are calculated.

7. Results are displayed on the graphical interface.

Input	RISC	CISC
ADD	LOAD, ADD, STORE	ADD
SUB	LOAD, SUB, STORE	SUB
MUL	LOAD, MUL, STORE	MUL
MOV	LOAD, STORE	MOV

5. Simulation Example

For the input: ADD R1,R2; SUB R3,R4; MUL R5,R6; MOV R1,R2

RISC execution contains 11 instructions in the simplified model. Estimated cycles: ADD = 3, SUB = 3, MUL = 3 and MOV = 2, giving 11 cycles.

CISC execution contains 4 instructions. Estimated cycles: ADD = 2, SUB = 2, MUL = 3 and MOV = 2, giving 9 cycles.

Note: These clock-cycle values are simplified assumptions for educational simulation and do not represent exact real-processor performance.

6. Features

- Assembly program input box.
- Run Simulation button.
- Load Example button.
- Clear button.
- Separate RISC and CISC execution logs.
- Instruction count and estimated clock-cycle display.
- Progress bars for cycle comparison.
- Result showing which architecture has fewer estimated cycles.

7. Technology and Requirements

Programming Language: Python

GUI Library: Tkinter

Additional Library: ttk

Requirements: Python 3.x and Tkinter. No external database or advanced framework is required.

8. Basic Code Structure

run_simulation() – Reads input, identifies instructions and creates RISC/CISC sequences.

show_results() – Displays counts, cycles, logs and comparison.

example() – Loads the predefined example.

clear_all() – Clears input and previous results.

root.mainloop() – Keeps the GUI running and accepts user interaction.

9. Result and Conclusion

The RISC vs CISC Simulation Tool demonstrates the basic difference between the two instruction set architecture approaches. It shows how the same operation can be represented using multiple simple instructions in the simplified RISC model and fewer complex instructions in the simplified CISC model. The tool also compares instruction counts and estimated clock cycles in a simple graphical interface.

The project can be extended by adding more instructions, register visualization, memory simulation and pipeline visualization.

10. Project Files

- risc_cisc_simulator.py
- README-1.txt
- requirements-3.txt